KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT OF

Job Application Tracking System for Students

Submitted by

SUGAIL THANVEER K

25BIT156

Under the Guidance of

R MUGILAN

Trainer Name

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**PROJECT TITLE"** submitted in partial fulfillment of the requirements for the course **[Course Name]** is a **record of my original work** carried out under the guidance of **[Trainer Name]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

SUGAIL THANVEER K

CERTIFICATE

This is to certify that the project entitled **"PROJECT TITLE"** is a **bona fide work** carried out by **[YOUR NAME] [Class/Batch]**, [**Department Name]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

A **Job Application Tracking System for Students** developed in Java is a software application designed to help students efficiently manage and monitor their job applications. The system allows users to store details such as company name, job role, application date, and application status (e.g., applied, shortlisted, rejected, or selected). It provides features like adding, updating, deleting, and viewing application records, making the job search process more organized and less stressful. Built using Java, the program typically uses object-oriented principles and may include a user-friendly interface with file or database storage for data persistence. Overall, it helps students track progress, stay updated, and improve their job application management.

1. INTRODUCTION

The **Job Application Tracking System for Students** is a Java-based application designed to help students organize and manage their job search process effectively. It enables users to keep track of multiple job applications by storing important details such as company name, job position, application date, and status. Using Java and object-oriented programming concepts, the system provides a simple and efficient way to add, update, and view application

records, helping students stay organized and improve their chances of success in the recruitment process.

1.1 Objective

Here are **short notes on the objectives** of a *Job Application Tracking System for Students (Java program)*:

- To help students **organize and manage job applications** in a systematic way.
- To provide an easy way to **store and retrieve application details** such as company name, role, and status.
- To enable users to **add, update, and delete records** efficiently.
- To track the **progress of each application** (applied, shortlisted, rejected, selected).
- To reduce confusion and **avoid missing important opportunities or deadlines**.
- To apply **object-oriented programming concepts in Java** for practical learning.
- To improve **time management and productivity** during the job search process.

1.2 Overview

Here are **short notes on the overview** of a *Job Application Tracking System for Students (Java program)*:

- The system is a **Java-based application** designed to simplify the process of tracking job applications.
- It allows students to **maintain records** of jobs they have applied for in an organized manner.
- Users can **store key details** such as company name, job role, application date, and status.
- The program provides basic operations like **adding, viewing, updating, and deleting records**.
- It is developed using **object-oriented programming (OOP) concepts** in Java.
- Data can be stored using **files or a simple database** for future access.
- The system may include a **user-friendly interface** (console-based or GUI).
- Overall, it helps students **track progress and manage their job search efficiently**.

1.3 Features

Here are **short notes on the features** of a *Job Application Tracking System for Students (Java program)*:

- **User-friendly interface** – Simple console-based or GUI interface for easy interaction.
- **Add application details** – Allows users to enter company name, job role, date, and status.
- **View records** – Displays all saved job applications in an organized format.
- **Update information** – Enables modification of existing application details.
- **Delete records** – Allows removal of unwanted or outdated entries.
- **Search functionality** – Helps find applications based on company name or job role.
- **Status tracking** – Tracks progress such as applied, shortlisted, rejected, or selected.
- **Data storage** – Saves data using files or a database for future access.
- **Error handling** – Ensures smooth functioning by handling invalid inputs.
- **OOP-based design** – Implements Java concepts like classes, objects, and methods.

2. PROGRAM ANALYSIS

The program processes data through operations like **adding, updating, searching, and deleting records**.

- It ensures proper **data organization and validation** to avoid errors and duplication.
- The system evaluates performance based on **speed, accuracy, and ease of use**.
- It considers different storage methods like **file handling or database integration** for data persistence.
- The design focuses on **scalability and flexibility**, allowing future enhancements such as notifications or analytics.

- Overall, the analysis ensures the program meets user requirements and improves **efficiency in job tracking**

- **2.1 Problem Definition**

Students often apply to multiple jobs and face difficulty in **tracking application details manually**.

- Important information like **company name, job role, and application status** may get lost or confused.
- There is no simple system to **monitor progress** of each application in one place.
- Managing data using traditional methods (notes or spreadsheets) can be **time-consuming and error-prone**.
- Students may **miss deadlines or follow-ups** due to lack of proper organization.
- Hence, there is a need for a **Java-based system** that helps store, manage, and track job applications efficiently.
- The problem is to design a system that is **simple, reliable, and user-friendly** for students.

2.2 Project Overview

Here are **short notes on Problem Overview** for a *Job Application Tracking System for Students (Java program)*:

- Students applying for multiple jobs often face challenges in **organizing and tracking their applications**.
- Important details such as **company name, job role, dates, and status** are difficult to manage manually.
- Lack of a proper system leads to **confusion, duplication of entries, and missed opportunities**.
- Existing manual methods are **inefficient, time-consuming, and prone to errors**.
- There is a need for a **simple and automated solution** to manage application data effectively.
- The system aims to provide a **centralized platform** for storing and updating job-related information.
- Overall, the overview highlights the necessity of a **Java-based tracking system** to improve organization and efficiency in the job search process.

2.3 Scopes and Limitations

Here are short notes on Problem Scope and Limitations for a *Job Application Tracking System for Students (Java program)*:

Scope

- **The system is designed to manage and track job applications for students in an organized way.**
- **It allows users to add, view, update, delete, and search application records.**
- **It stores important details like company name, job role, date, and application status.**
- **The program can be implemented as a console-based or GUI-based Java application.**
- **It helps improve efficiency, organization, and time management during the job search process.**

Limitations

- **The system may have limited features compared to professional job portals.**
- **It may not support real-time updates or notifications unless extended.**
- **Data storage might be basic (file-based) and less secure without advanced databases.**
- **The application is usually designed for single-user use and may not support multiple users.**
- **It depends on manual data entry, which can still lead to input errors.**

3. SYSTEM DESIGN

Here are **short notes on System Design** for a *Job Application Tracking System for Students (Java program)*:

- The system follows a **modular design approach**, dividing the application into different components such as user interface, logic, and data storage.
- It uses **object-oriented programming (OOP)** concepts like classes and objects to represent job applications and operations.
- A main class controls the program flow, while separate classes handle **application data and functionalities**.
- The system includes modules for **adding, viewing, updating, deleting, and searching records**.

- Data is stored using **file handling or a simple database** for persistence.
- The user interface can be **console-based or GUI-based (using Swing/JavaFX)**.
- Input validation and error handling are included to ensure **data accuracy and smooth operation**.
- Overall design focuses on **simplicity, usability, and easy maintenance**.

3.1 Modules

Here are **short notes on Modules** for a *Job Application Tracking System for Students (Java program)*:

1. **User Interface Module**
    - Provides interaction with the user.
    - Displays menus and options for managing job applications.
2. **Add Application Module**
    - Allows users to input new job application details.
    - Stores information like company name, job role, date, and status.
3. **View Applications Module**
    - Displays all saved job applications in an organized format.
4. **Update Application Module**
    - Enables editing of existing application records.
    - Updates status or other details as required.
5. **Delete Application Module**
    - Removes unwanted or outdated job application records.
6. **Search Module**
    - Allows searching of applications by company name or job role.
7. **Data Storage Module**
    - Handles saving and retrieving records using **files or database**.
8. **Validation and Error Handling Module**
    - Ensures input data is correct and prevents invalid operations.

This modular structure makes the system **organized, scalable, and easy to maintain**.

4. METHODOLOGY

Here are the topic about the methodology

- Problem Analysis

- System Design

- Program Development

- Testing and Validation

- Result and Documentation

- **Problem Analysis**: Identify the challenges students face in managing multiple job applications and define the system requirements.
- **System Design**: Plan the structure using **object-oriented concepts**, creating modules for adding, viewing, updating, deleting, and searching applications.
- **Program Development**: Implement the system in **Java**, integrating all modules and using **file handling or a database** for storing data.
- **Testing and Validation**: Check each module for **accuracy, reliability, and error handling**, ensuring correct input and output.
- **Result and Documentation**: Confirm the system effectively manages applications and prepare **user-friendly documentation** explaining features and usage.

5. SOURCE CODE

Include important parts of the Java code such as:

- Main class
- Menu logic

- Billing calculation

Example:

```
import java.util.*;

// Job Application Class
class JobApplication {
    private int id;
    private String companyName;
    private String role;
    private String status; // Applied, Interview, Rejected, Offer

    public JobApplication(int id, String companyName, String role, String
status) {
        this.id = id;
        this.companyName = companyName;
        this.role = role;
        this.status = status;
    }

    public int getId() {
        return id;
    }

    public String getCompanyName() {
        return companyName;
    }

    public String getRole() {
        return role;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

    public void display() {
        System.out.println("ID: " + id +
                ", Company: " + companyName +
                ", Role: " + role +
                ", Status: " + status);
    }
}
```

```java
// Main System Class
public class JobT0rackingSystem {
  private static List<JobApplication> applications = new ArrayList<>();
  private static Scanner scanner = new Scanner(System.in);
  private static int idCounter = 1;

  public static void main(String[] args) {
    while (true) {
      System.out.println("\n=== Job Application Tracking System ===");
      System.out.println("1. Add Application");
      System.out.println("2. View Applications");
      System.out.println("3. Search by Company");
      System.out.println("4. Update Status");
      System.out.println("5. Delete Application");
      System.out.println("6. Exit");
      System.out.print("Choose option: ");

      int choice = scanner.nextInt();
      scanner.nextLine(); // consume newline

      switch (choice) {
        case 1:
          addApplication();
          break;
        case 2:
          viewApplications();
          break;
        case 3:
          searchApplication();
          break;
        case 4:
          updateStatus();
          break;
        case 5:
          deleteApplication();
          break;
        case 6:
          System.out.println("Exiting...");
          return;
        default:
          System.out.println("Invalid choice!");
      }
    }
  }

  private static void addApplication() {
    System.out.print("Enter Company Name: ");
    String company = scanner.nextLine();
```

```
        System.out.print("Enter Role: ");
        String role = scanner.nextLine();

        System.out.print("Enter Status (Applied/Interview/Rejected/Offer): ");
        String status = scanner.nextLine();

        JobApplication app = new JobApplication(idCounter++, company, role,
status);
        applications.add(app);

        System.out.println("Application Added Successfully!");
    }

    private static void viewApplications() {
        if (applications.isEmpty()) {
            System.out.println("No applications found.");
            return;
        }

        for (JobApplication app : applications) {
            app.display();
        }
    }

    private static void searchApplication() {
        System.out.print("Enter Company Name to search: ");
        String search = scanner.nextLine();

        boolean found = false;
        for (JobApplication app : applications) {
            if (app.getCompanyName().equalsIgnoreCase(search)) {
                app.display();
                found = true;
            }
        }

        if (!found) {
            System.out.println("No applications found for this company.");
        }
    }

    private static void updateStatus() {
        System.out.print("Enter Application ID: ");
        int id = scanner.nextInt();
        scanner.nextLine();

        for (JobApplication app : applications) {
```

```
        if (app.getId() == id) {
            System.out.print("Enter New Status: ");
            String newStatus = scanner.nextLine();
            app.setStatus(newStatus);
            System.out.println("Status Updated!");
            return;
        }
    }

    System.out.println("Application not found.");
  }

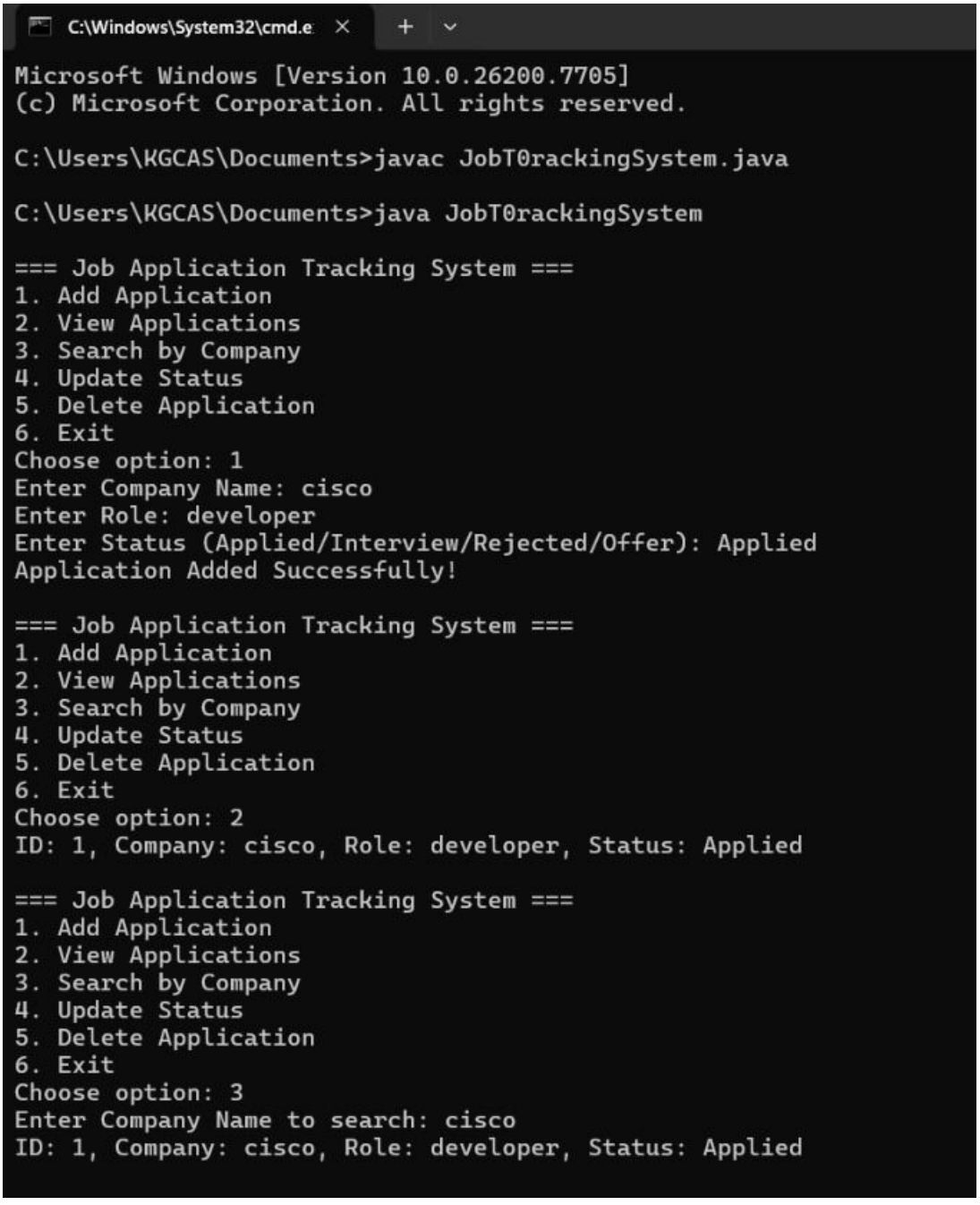
  private static void deleteApplication() {
    System.out.print("Enter Application ID to delete: ");
    int id = scanner.nextInt();

    Iterator<JobApplication> iterator = applications.iterator();
    while (iterator.hasNext()) {
        JobApplication app = iterator.next();
        if (app.getId() == id) {
            iterator.remove();
            System.out.println("Application Deleted!");
            return;
        }
    }

    System.out.println("Application not found.");
  }
}
```

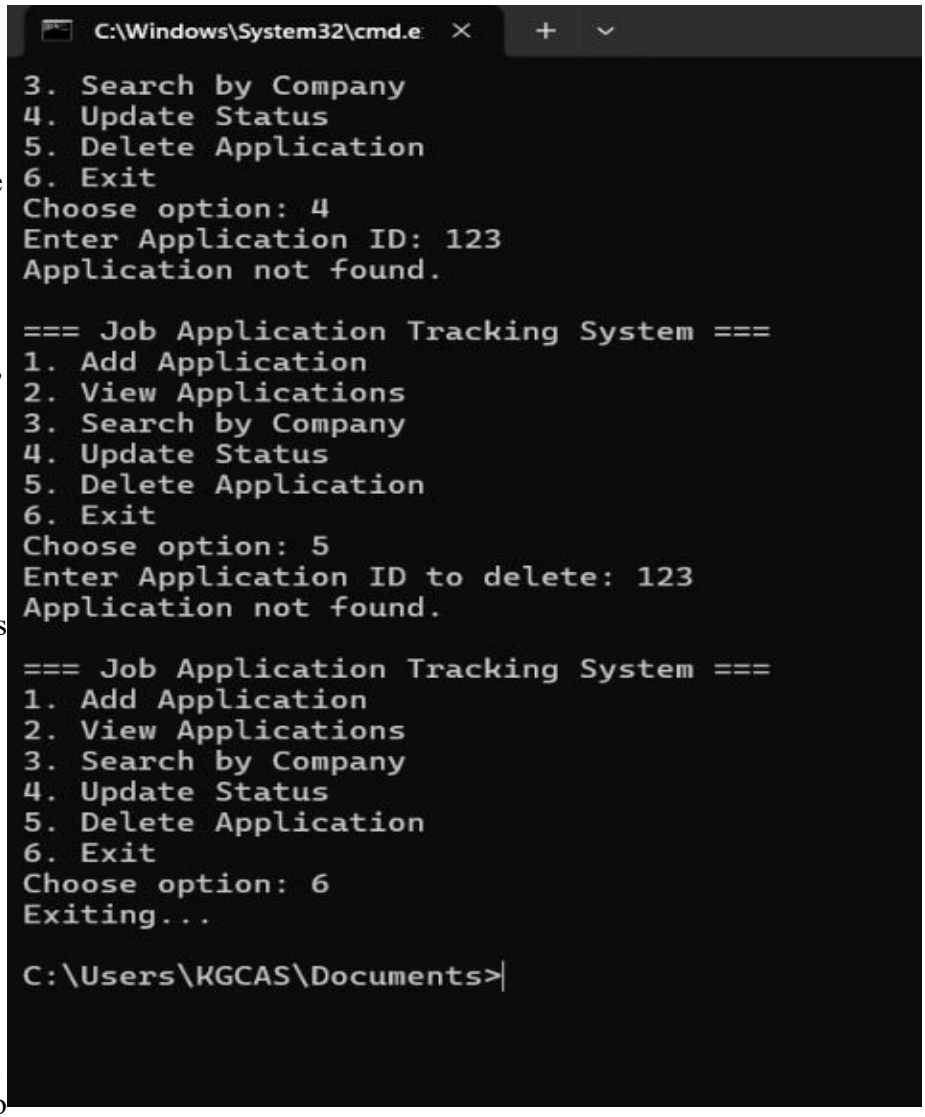
6. INPUT AND OUTPUT SCREENS

7.
8.


C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.
C:\Users\KGCAS\Documents>javac JobT0rackingSystem.java
C:\Users\KGCAS\Documents>java JobT0rackingSystem
=== Job Application Tracking System ===
1. Add Application
2. View Applications
3. Search by Company
4. Update Status
5. Delete Application
6. Exit
Choose option: 1
Enter Company Name: cisco
Enter Role: developer
Enter Status (Applied/Interview/Rejected/Offer): Applied
Application Added Successfully!
=== Job Application Tracking System ===
1. Add Application
2. View Applications
3. Search by Company
4. Update Status
5. Delete Application
6. Exit
Choose option: 2
ID: 1, Company: cisco, Role: developer, Status: Applied
=== Job Application Tracking System ===
1. Add Application
2. View Applications
3. Search by Company
4. Update Status
5. Delete Application
6. Exit
Choose option: 3
Enter Company Name to search: cisco
ID: 1, Company: cisco, Role: developer, Status: Applied

9. DISCUSSION & CONCLUSION

7.1 Discussion



The **Hotel & Restaurant Billing System** was developed to automate the billing process in restaurants and hotels. The system allows users to select food items, enter quantities, and generate bills automatically.

During the development process, Core Java programming concepts such as loops, conditional statements, methods, and object-oriented principles were applied. The system successfully performs bill calculations and generates accurate invoices.

The system was tested with different inputs to ensure that it performs the required calculations correctly. The results show that the system reduces manual effort and improves billing efficiency.

7.2 Conclusion

The **Hotel & Restaurant Billing System** successfully demonstrates the practical application of Core Java programming concepts in developing a real-world business application.

The system automates the billing process, reduces calculation errors, and provides faster service to customers. It simplifies restaurant billing operations by generating accurate invoices based on customer orders.

This project helps students understand how Java programming can be used to develop practical software systems. The project can be further enhanced by adding features such as database connectivity, graphical user interfaces, and online payment systems.

10.REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com